

27. live 計測鮮度監視

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	6
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 rclpy、rcl、rmw、DDS/RTPS の層	7
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	8
6.11 rqt_graph、ros2 CLI、rosbag2 をいつ使うか	8
6.12 制御、状態、入力、モデル予測制御 MPC	9
7 関数・クラスを上から順に解説	9
7.1 L12 クラス SolarPreflightNode	9

7.2	L15 関数 SolarPreflightNode.__init__	11
7.3	L41 関数 SolarPreflightNode._seen	12
7.4	L42 関数 SolarPreflightNode._seen.callback	13
7.5	L47 関数 SolarPreflightNode._required	14
7.6	L59 関数 SolarPreflightNode._step	15
7.7	L78 関数 main	16
7.8	設定値と入力パラメータ	17
7.9	ROS topic I/O	17
8	処理の流れ	17
9	このファイルの位置づけ	17

1 対象

項目	内容
ファイル	mpc_solarcar/solar_preflight_node.py
ソース SHA-256	d1c39ce9b2c8e6006ec42892c09b8ede984748d44555a5f330607b92aa5a4d9
種別	Python
区分	runtime node
役割	speed、distance、battery、planner status の鮮度を見て system/state と health を publish する。
起動文脈	起動可否と運用中の健全性の監視役。

2 このファイルを読む理由

speed、distance、battery、planner status の鮮度を見て system/state と health を publish する。

- planner 自体は止めずに、system/state と health を出す。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/live_launch.py
- launch/solar_measurement.launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/solar_preflight_logic.py
- mpc_solarcar/speed_command_bridge_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/solar_preflight_logic.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from .solar_preflight_logic import evaluate_freshness

class SolarPreflightNode(Node):
    """Preflight monitor based only on solar-car telemetry freshness."""

    def __init__(self):
        super().__init__("solar_preflight_node")
```

```

self.declare_parameter("startup_grace_sec", 10.0)
self.declare_parameter("measurement_timeout_sec", 3.0)
self.declare_parameter("require_speed", True)
self.declare_parameter("require_distance", True)
self.declare_parameter("require_battery", True)
self.declare_parameter("require_planner", True)

self.start_mono = time.monotonic()
self.last_seen: dict[str, float] = {}

self.pub_state = self.create_publisher(String, "/system/state", 10)
self.pub_health = self.create_publisher(Float32, "/system/health", 10)
self.pub_diag = self.create_publisher(String, "/system/diag", 10)

```

5 主要構造の読み取り

主要クラスは SolarPreflightNode。主要関数は callback, main。ROS パラメータ宣言は 6 件。ROS I/O は publisher 3 件、subscription 6 件。

5.1 主要クラス・関数

行	種別	名前	補足
12	class	SolarPreflightNode	bases: Node
15	function	__init__	args: self
41	function	_seen	args: self, key
42	function	callback	args: _msg
47	function	_required	args: self
59	function	_step	args: self
78	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
12	クラス SolarPreflightNode を定義する。
78	関数 main を定義する。
88	条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	from __future__ import annotations	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	import time	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L24, L43, L60。

5	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L79, L82, L85。
6	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L12。
7	<code>from std_msgs. msg import Float32, Float32MultiArray, String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L27, L28, L29, L31, L32, L33, L34, L35, ...。
9	<code>from solar_preflight_ logic import evaluate_ freshness</code>	preflight 判定ロジックから <code>evaluate_freshness</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_preflight_logic.py</code> 。このファイル内での主な使用位置は L66。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。`'x = f()'`では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

`'float(...)'`、`'int(...)'`、`'bool(...)'` は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

`'None'` は値が無いことを示す単一のオブジェクト、`'math.nan'` は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、`'is None'` と `'math.isfinite'` を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self.step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は MultiThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```

ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock

```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.12 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L12 クラス SolarPreflightNode

- 行範囲: L12–L75
- 所属: module 直下
- 定義: `SolarPreflightNode(bases=Node)`
- docstring: Preflight monitor based only on solar-car telemetry freshness.
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `docstring` または説明文字列を置く。
2. 関数 `__init__` を定義する。
3. 関数 `_seen` を定義する。
4. 関数 `_required` を定義する。
5. 関数 `_step` を定義する。

代表コード断片

```
class SolarPreflightNode(Node):
    """Preflight monitor based only on solar-car telemetry freshness."""

    def __init__(self):
        super().__init__("solar_preflight_node")
        self.declare_parameter("startup_grace_sec", 10.0)
        self.declare_parameter("measurement_timeout_sec", 3.0)
        self.declare_parameter("require_speed", True)
        self.declare_parameter("require_distance", True)
        self.declare_parameter("require_battery", True)
        self.declare_parameter("require_planner", True)

        self.start_mono = time.monotonic()
        self.last_seen: dict[str, float] = {}

        self.pub_state = self.create_publisher(String, "/system/state", 10)
        self.pub_health = self.create_publisher(Float32, "/system/health", 10)
        self.pub_diag = self.create_publisher(String, "/system/diag", 10)

        self.create_subscription(Float32, "/vehicle/speed_kmh", self._seen("speed"), 10)
        self.create_subscription(Float32, "/vehicle/s_kmh", self._seen("distance"), 10)
        self.create_subscription(Float32, "/vehicle/batt_soc", self._seen("batt_soc"), 10)
        self.create_subscription(Float32, "/vehicle/batt_voltage_v", self._seen("batt_voltage"), 10)
        self.create_subscription(Float32, "/vehicle/batt_current_a", self._seen("batt_current"), 10)
        self.create_subscription(Float32MultiArray, "/planner/status", self._seen("planner"), 10)

        self.timer = self.create_timer(1.0, self._step)
        self.get_logger().info("SolarPreflightNode started.")

    def _seen(self, key: str):
        def callback(_msg):
```

```
        self.last_seen[key] = time.monotonic()

    return callback
```

...

7.2 L15 関数 SolarPreflightNode.__init__

- 行範囲: L15–L39
- 所属: SolarPreflightNode
- 定義: `__init__(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `__init__`, `_seen`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `get_logger`, `info`, `monotonic`, `super`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._seen`, `self._step`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.get_logger`
- 更新する主なインスタンス属性: `self.last_seen`, `self.pub_diag`, `self.pub_health`, `self.pub_state`, `self.start_mono`, `self.timer`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.start_mono` に `time.monotonic()` の結果を代入する。
9. `self.last_seen` に `{}` を代入する。
10. `self.pub_state` に `self.create_publisher(String, '/system/state', 10)` の結果を代入する。
11. `self.pub_health` に `self.create_publisher(Float32, '/system/health', 10)` の結果を代入する。

12. self.pub_diag に self.create_publisher(String, '/system/diag', 10) の結果を代入する。
13. self.create_subscription(...) を実行する。
14. self.create_subscription(...) を実行する。
15. self.create_subscription(...) を実行する。
16. self.create_subscription(...) を実行する。
17. self.create_subscription(...) を実行する。
18. self.create_subscription(...) を実行する。
19. self.timer に self.create_timer(1.0, self._step) の結果を代入する。
20. self.get_logger().info(...) を実行する。

代表コード断片

```
def __init__(self):
    super().__init__("solar_preflight_node")
    self.declare_parameter("startup_grace_sec", 10.0)
    self.declare_parameter("measurement_timeout_sec", 3.0)
    self.declare_parameter("require_speed", True)
    self.declare_parameter("require_distance", True)
    self.declare_parameter("require_battery", True)
    self.declare_parameter("require_planner", True)

    self.start_mono = time.monotonic()
    self.last_seen: dict[str, float] = {}

    self.pub_state = self.create_publisher(String, "/system/state", 10)
    self.pub_health = self.create_publisher(Float32, "/system/health", 10)
    self.pub_diag = self.create_publisher(String, "/system/diag", 10)

    self.create_subscription(Float32, "/vehicle/speed_kmh", self._seen("speed"), 10)
    self.create_subscription(Float32, "/vehicle/s_kmh", self._seen("distance"), 10)
    self.create_subscription(Float32, "/vehicle/batt_soc", self._seen("batt_soc"), 10)
    self.create_subscription(Float32, "/vehicle/batt_voltage_v", self._seen("batt_voltage"), 10)
    self.create_subscription(Float32, "/vehicle/batt_current_a", self._seen("batt_current"), 10)
    self.create_subscription(Float32MultiArray, "/planner/status", self._seen("planner"), 10)

    self.timer = self.create_timer(1.0, self._step)
    self.get_logger().info("SolarPreflightNode started.")
```

7.3 L41 関数 SolarPreflightNode._seen

- 行範囲: L41–L45
- 所属: SolarPreflightNode
- 定義: `_seen(self, key: str)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `monotonic`
- 戻り値の要点: `callback`
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: `self.last_seen`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 関数 `callback` を定義する。
2. `callback` を返す。

代表コード断片

```
def _seen(self, key: str):
    def callback(_msg):
        self.last_seen[key] = time.monotonic()

    return callback
```

7.4 L42 関数 `SolarPreflightNode._seen.callback`

- 行範囲: L42–L43
- 所属: `SolarPreflightNode._seen`
- 定義: `callback(_msg)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `monotonic`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.last_seen`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `self.last_seen[key]` に `time.monotonic()` の結果を代入する。

代表コード断片

```
def callback(_msg):  
    self.last_seen[key] = time.monotonic()
```

7.5 L47 関数 SolarPreflightNode._required

- 行範囲: L47–L57
- 所属: SolarPreflightNode
- 定義: `_required(self) -> tuple[str, ...]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `bool`, `extend`, `get_parameter`, `tuple`
- 戻り値の要点: `tuple(required)`
- 主なローカル代入名: `required`
- 読み取る主なインスタンス属性: `self.get_parameter`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `required` に `[]` を代入する。
2. 条件 `bool(self.get_parameter('require_speed').value)` を判定し、真なら内部処理を行う。
3. `required.append(...)` を実行する。
4. 条件 `bool(self.get_parameter('require_distance').value)` を判定し、真なら内部処理を行う。
5. `required.append(...)` を実行する。
6. 条件 `bool(self.get_parameter('require_battery').value)` を判定し、真なら内部処理を行う。
7. `required.extend(...)` を実行する。
8. 条件 `bool(self.get_parameter('require_planner').value)` を判定し、真なら内部処理を行う。
9. `required.append(...)` を実行する。
10. `tuple(required)` を返す。

代表コード断片

```
def _required(self) -> tuple[str, ...]:
    required: list[str] = []
    if bool(self.get_parameter("require_speed").value):
        required.append("speed")
    if bool(self.get_parameter("require_distance").value):
        required.append("distance")
    if bool(self.get_parameter("require_battery").value):
        required.extend(("batt_soc", "batt_voltage", "batt_current"))
    if bool(self.get_parameter("require_planner").value):
        required.append("planner")
    return tuple(required)
```

7.6 L59 関数 SolarPreflightNode._step

- 行範囲: L59–L75
- 所属: SolarPreflightNode
- 定義: `_step(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `String`, `_required`, `evaluate_freshness`, `float`, `get_parameter`, `monotonic`, `publish`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `ages`, `key`, `now`, `required`, `result`
- 読み取る主なインスタンス属性: `self._required`, `self.get_parameter`, `self.last_seen`, `self.pub_diag`, `self.pub_health`, `self.pub_state`, `self.start_mono`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `now` に `time.monotonic()` の結果を代入する。
2. `required` に `self._required()` の結果を代入する。
3. `ages` に `{key: now - self.last_seen[key] if key in self.last_seen else None for key in required}` の結果を代入する。
4. `result` に `evaluate_freshness(elapsed_sec=now - self.start_mono, ages_sec=ages, required=required, timeout_sec=float(self.get_parameter('measurement_timeout_sec').value), startup_grace_sec=float(self.get_parameter('startup_grace_sec').value))` の結果を代入する。
5. `self.pub_state.publish(...)` を実行する。
6. `self.pub_health.publish(...)` を実行する。
7. `self.pub_diag.publish(...)` を実行する。

代表コード断片

```
def _step(self):
    now = time.monotonic()
    required = self._required()
    ages = {
        key: (now - self.last_seen[key]) if key in self.last_seen else None
        for key in required
    }
    result = evaluate_freshness(
        elapsed_sec=now - self.start_mono,
        ages_sec=ages,
        required=required,
        timeout_sec=float(self.get_parameter("measurement_timeout_sec").value),
        startup_grace_sec=float(self.get_parameter("startup_grace_sec").value),
    )
    self.pub_state.publish(String(data=result.state))
    self.pub_health.publish(Float32(data=float(result.health)))
    self.pub_diag.publish(String(data=result.diagnostic))
```

7.7 L78 関数 main

- 行範囲: L78–L85
- 所属: module 直下
- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `SolarPreflightNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `rcipy.init(...)` を実行する。
2. `node` に `SolarPreflightNode()` の結果を代入する。
3. 例外処理を伴う `try` ブロックを実行する。
4. `rcipy.spin(...)` を実行する。
5. 成否にかかわらず `finally` で:
6. `node.destroy_node(...)` を実行する。
7. `rcipy.shutdown(...)` を実行する。

代表コード断片

```
def main():
    rclpy.init()
    node = SolarPreflightNode()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

7.8 設定値と入力パラメータ

行	名前	既定値・補足
17	startup_grace_sec	10.0
18	measurement_timeout_sec	3.0
19	require_speed	True
20	require_distance	True
21	require_battery	True
22	require_planner	True

7.9 ROS topic I/O

行	種別	topic	callback / 補足
27	publisher	/system/state	publisher
28	publisher	/system/health	publisher
29	publisher	/system/diag	publisher
31	subscription	/vehicle/speed_kmh	self._seen('speed')
32	subscription	/vehicle/s_kmh	self._seen('distance')
33	subscription	/vehicle/batt_soc	self._seen('batt_soc')
34	subscription	/vehicle/batt_voltage_v	self._seen('batt_voltage')
35	subscription	/vehicle/batt_current_a	self._seen('batt_current')
36	subscription	/planner/status	self._seen('planner')

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_preflight_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。